

SportFlow: Plataforma Distribuída de E-Commerce com Spring Boot e React

Diogo Dinis
Dep. de Informática
Universidade da Beira Interior
diogo.lopes.dinis@ubi.pt

João Rodrigues
Dep. de Informática
Universidade da Beira Interior
joao.ff.rodrigues@ubi.pt

Maria Machado
Dep. de Informática
Universidade da Beira Interior
maria.rodrigues.machado@ubi.pt

Resumo—Este documento descreve a implementação da plataforma "SportFlow", uma aplicação *web* orientada para o comércio eletrónico de artigos desportivos, desenvolvida no âmbito da unidade curricular de Sistemas Distribuídos. O sistema foi concebido adotando o paradigma *API-First*, estabelecendo uma separação entre o cliente e o servidor. O servidor (*backend*) foi construído em Java com a *Spring Boot Framework*, implementando uma API RESTful segura e escalável, com persistência de dados em MySQL (gerida por *Spring Data JPA* e *Flyway*). A interface de utilizador (*frontend*) consiste numa *Single Page Application* (SPA) reativa desenvolvida em React. A comunicação é protegida através de autenticação *stateless* baseada em *JSON Web Tokens* (JWT) transmitidos via *Cookies HttpOnly*. A infraestrutura conta ainda com integração contínua (CI/CD) via *GitHub Actions* e orquestração de contentores via *Docker Compose*.

Index Terms—Sistemas Distribuídos, Spring Boot, React, JWT, HttpOnly, E-commerce, CI/CD, Docker.

I. INTRODUÇÃO

No cenário tecnológico atual, a prática do desenvolvimento de aplicações *web* é feita segundo um princípio de separação *frontend-backend*. A dissociação entre o cliente (responsável pela apresentação interativa) e o servidor (responsável pela segurança, armazenamento e lógica de negócio) permite alcançar maior escalabilidade, facilitar a manutenção, e promover a reutilização de serviços de forma distribuída.

O presente trabalho materializa estas práticas no desenvolvimento da plataforma "SportFlow". O domínio de atuação assenta na venda de equipamento desportivo, permitindo aos utilizadores navegar num catálogo estruturado em categorias. Para além do circuito tradicional de compra, a aplicação engloba funcionalidades que almejam a retenção de utilizadores, tais como Listas de Desejos (*Wishlists*) sincronizadas, Avaliações de Produtos (*Reviews*) com classificação por estrelas, e um *Backoffice* de administração com extração de métricas de vendas e controlo de inventário em tempo real.

O relatório organiza-se da seguinte forma: a Secção II descreve a arquitetura geral do sistema; a Secção III explora o modelo de persistência; a Secção IV foca-se na segurança e mitigação de vulnerabilidades; a Secção V fornece demonstrações e casos de uso da interface; a Secção VI aborda infraestrutura e testes automáticos; e, por fim, a Secção VII sintetiza as conclusões.

II. ARQUITETURA DO SISTEMA

A topologia da plataforma SportFlow segue o paradigma *API-First*, dividindo o sistema em nós distribuídos comunicando assincronamente via HTTP.

A. Camada de Servidor (Backend)

O processamento central reside num servidor desenvolvido em Java 21, de *framework* Spring Boot. A sua constituição obedece ao padrão Model-View-Controller (adaptado para o fornecimento de APIs), subdividido em:

- **Controladores REST:** Interceptam pedidos HTTP do exterior, extraem parâmetros, invocam lógicas aplicacionais e devolvem as respostas encapsuladas em *Data Transfer Objects* (DTOs).
- **Serviços de Negócio (Services):** Garantem transacionalidade e o cumprimento de regras complexas, desde o serviço de autenticação até aos serviços de *Wishlist* e Cálculo de Estatísticas.
- **Repositórios:** Interfaces *Spring Data JPA* que atuam como abstração comunicacional de baixo nível (*Hibernate*) com a base de dados relacional.

B. Camada de Cliente (Frontend)

O cliente assume a forma de uma SPA reativa construída com React 18 (*TypeScript*) e a ferramenta *Vite*. O cliente foca-se na gestão do estado da interface distribuído em *Contextos React* (ex: Auth, Cart, Wishlist). Os pedidos de rede são geridos utilizando a biblioteca *Axios*, garantindo uma navegação fluida sem recarregamentos de página (recorrendo a mais de 12 páginas modulares de *Routing*).

III. MODELO DE DADOS E PERSISTÊNCIA

A persistência central foi construída numa base de dados relacional **MySQL 8.0**. A construção estrutural e o preenchimento de dados demonstrativos estão implementados pela ferramenta *Flyway*, que executa as migrações SQL no momento do arranque da aplicação.

O esquema suporta 9 tabelas fundamentais, destacando-se:

- 1) **User/Customer:** Identidade do utilizador, perfil de autorização e cifra criptográfica irreversível da *password*.

- 2) **Product e Category:** A entidade `Product` restringe inserções negativas e implementa *soft-delete* (arquivamento lógico) para preservar a integridade do catálogo e faturas.
- 3) **Sale e SaleItem:** A agregação de uma venda (`Sale`) e as suas linhas de fatura congeladas no tempo (`SaleItem`). Isto impossibilita que alterações contemporâneas ao preço de um produto corrompam as métricas de faturas antigas.
- 4) **Reviews e Wishlist:** Tabelas associativas que ligam os utilizadores aos produtos, suportando a lógica de negócio de classificação social da loja.

IV. SEGURANÇA, AUTENTICAÇÃO E PREVENÇÃO XSS

Um aspeto crítico de qualquer solução *e-commerce* prende-se com a proteção de acessos. A opção pelo módulo *Spring Security* com encriptação de senhas via *BCrypt* serviu de base. Dado o caráter *stateless* das arquiteturas RESTful, usámos **JSON Web Tokens (JWT)** para uma autorização descentralizada.

No entanto, para maximizar a segurança da aplicação face a vetores de ataque comuns, o tratamento do JWT teve que ser otimizado:

- **Cookies HttpOnly:** O JWT não é injetado manualmente em código JavaScript (o que o exporia a ataques de *Cross-Site Scripting* - XSS). O servidor retorna o *token* embutido num Cookie assinalado com a diretiva `HttpOnly`.
- **Injeção Automática:** O *browser* do cliente encarrega-se, de forma autónoma e "cega", de anexar o Cookie em cada transação HTTP com o domínio do *backend*.
- O filtro global da infraestrutura *Spring* interceta as chamadas, descripta o Cookie, valida a assinatura criptográfica e autoriza a operação baseada nas *roles* (ex: rotas POST para produtos fechadas exclusivamente a perfis ADMIN).

V. DEMONSTRAÇÃO E CASOS DE USO

A verificação prática dos requisitos é consubstanciada nas capturas de ecrã documentadas, refletindo o ecossistema de funcionalidades do cliente:

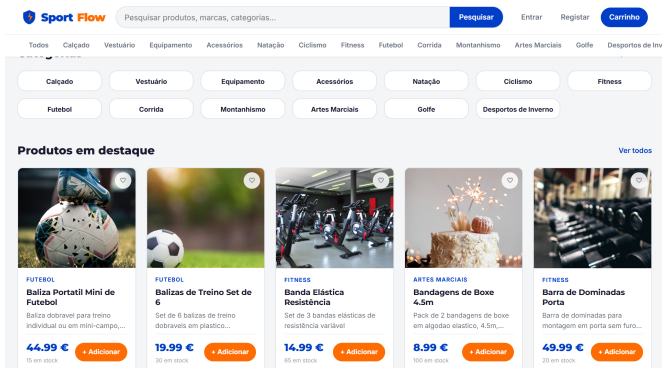


Figura 1. Interface do Catálogo, com sistema de ordenação, pesquisa textual e adição imediata a Lista de Desejos.

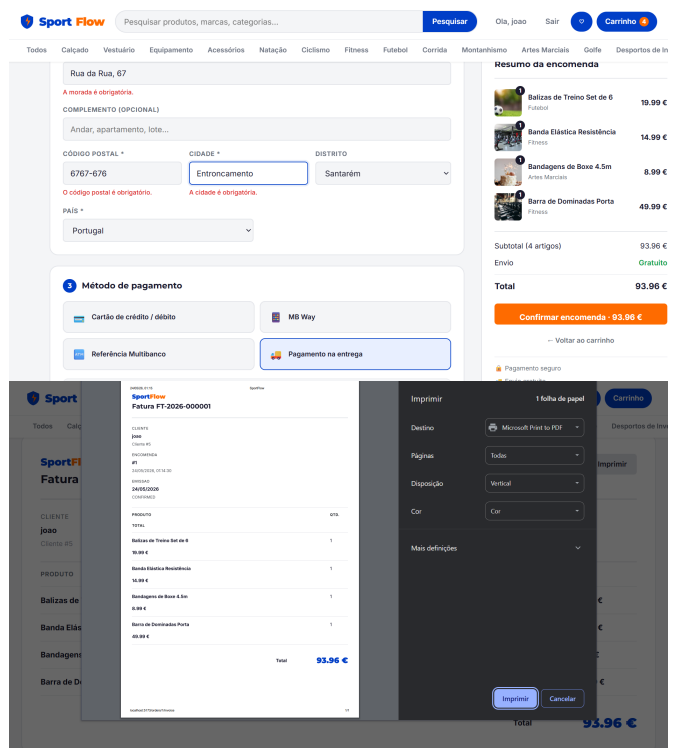


Figura 2. Processo de *Checkout*, onde os itens do *localStorage* do cliente sofrem validação autêntica de *stock* no servidor, originando uma Fatura.

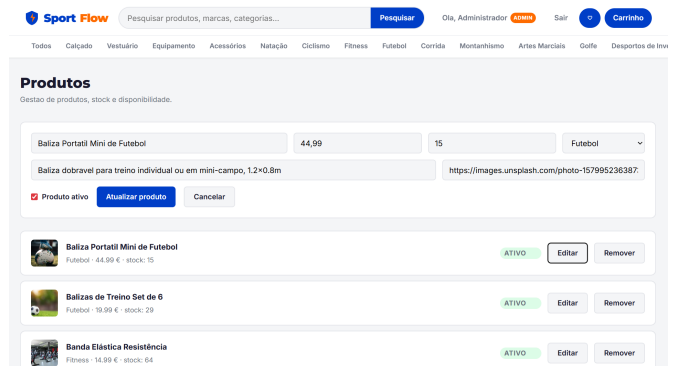


Figura 3. Painel de Administração (Backoffice) com acesso restrito a dashboards estatísticos de lucro e atividade comercial da plataforma.

VI. INFRAESTRUTURA, DEVOPS E CI/CD

Elevando a qualidade de engenharia do projeto, a organização do sistema foge aos moldes puramente académicos, adotando práticas profissionais:

- **Testes Unitários (Mockito & H2):** A verificação das regras cruciais (como concorrência em reduções de inventário) foi garantida por metodologias de teste exaustivo com *Mockito*, correndo de forma efémera numa base de dados *H2* em memória, que anula a latência de acesso físico.
- **Integração Contínua (GitHub Actions):** Configuração de **workflows** *CI/CD* que, perante qualquer *Push* no

repositório remoto, despoletam uma bateria automatizada de testes e um *build* da infraestrutura Java (versão 21) impulsionado por *Maven*.

- **Orquestração de Contentores:** O ambiente local de demonstração e testes é unificado pelo **Docker Compose**, isolando em contentores dependentes as 3 peças essenciais do sistema: A Base de Dados (MySQL 8.0), o Backend (Spring Boot), e o Frontend (React/Vite).

VII. CONCLUSÃO

A materialização da plataforma distribuída SportFlow constituiu um exercício de excelência na adoção e consolidação de metodologias modernas na Engenharia de Software. A articulação pragmática de ecossistemas consagrados no mercado — Spring Boot, React, MySQL, e Docker — providenciou o nível exigível de agilidade aplicacional e resiliência informacional.

As limitações de *APIs stateless* foram superadas com implementações seguras de JWT em *Cookies HttpOnly*, garantindo que a separação entre camadas de apresentação e serviços não comprometeu a inviolabilidade. O projeto valida a capacidade técnica de desenvolver sistemas ricos (como Sistemas de Faturação, *Reviews* e Carrinhos persistentes), tolerantes a falhas e prontos para um posterior desenvolvimento sustentado por práticas de Integração Contínua.

REFERÊNCIAS

- [1] Universidade da Beira Interior, “Sistemas Distribuídos - Materiais e Recursos,” [Online]. Disponível em: <https://moodle.ubi.pt/>.
- [2] VMware, Inc., “Spring Framework Reference Documentation,” [Online]. Disponível em: <https://spring.io/projects/spring-framework>.
- [3] Meta Platforms, Inc., “React Documentation,” [Online]. Disponível em: <https://react.dev/>.
- [4] Docker Inc., “Docker Documentation,” [Online]. Disponível em: <https://docs.docker.com/>.